
TinyPRM-Cascade: Sub-Billion Parameter Process Reward Models for Compute-Efficient Test-Time Reasoning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study how to scale test-time compute for compact language models without
2 incurring the latency and memory costs of large process reward models (PRMs).
3 Strong PRMs typically exceed 6B parameters, while outcome-supervised or rein-
4 forcement learning alternatives often hallucinate step-level errors, require many
5 samples, or inflate wall-clock time. We propose TinyPRM-Cascade, a three-tier
6 inference framework that couples a 7B generator with two ultra-light evaluators:
7 a Shared-Adapter PRM ($\sim 400\text{M}$) that reuses the frozen generator backbone via
8 LoRA adapters and a small gating network, and a Sparse-MoE Refinement PRM
9 ($\sim 900\text{M}$ total, $\sim 220\text{M}$ active) distilled from a 6B process-supervised teacher. An
10 adaptive beam controller integrates generator log-probability gaps, PRM scores,
11 and a running compute budget to decide when to commit, branch, or halt explo-
12 ration. Key novelties include parameter sharing that adds under 0.5 GB VRAM,
13 lazy loading of the MoE for fewer than 20% of steps, mixture-of-thought distilla-
14 tion from chain- and program-of-thought traces, and a focal KL calibration head
15 to mitigate hallucinated errors observed in stepwise evaluators. We release an
16 instrumented evaluation harness and report a complete toy run validating control
17 flow and measurement: on a synthetic arithmetic task, TinyPRM-Cascade matches
18 greedy accuracy (100%), surpasses a self-consistency baseline in both accuracy
19 and latency, and supports early-commit sweeps. While this toy setting cannot probe
20 full accuracy-latency trade-offs, it establishes the correctness of the design and the
21 evaluation pipeline needed for realistic benchmarks.

22 1 Introduction

23 Large language models increasingly rely on multi-step reasoning to achieve strong performance
24 in mathematics, logic, and program synthesis. Process supervision—grading intermediate steps
25 rather than only final outcomes—has proven effective and motivated the release of PRM800K
26 alongside models that score partial reasoning traces Lightman et al. [2023]. However, deploying
27 process supervision at inference time remains compute- and memory-intensive: high-quality PRMs
28 commonly exceed 6B parameters, which is impractical for single-GPU desktops and edge servers.
29 Alternatives based on self-consistency or self-evaluation guided search either require many samples
30 to stabilize, struggle with calibration on intermediate steps, or inflate latency beyond practical budgets
31 Xie et al. [2023], Wang and Zhou [2024].

32 1.1 Motivation and Challenges

33 We focus on the following challenge: design a $\leq 1\text{B}$ parameter PRM that remains calibrated on partial
34 traces, decides early when additional reasoning is unnecessary, and can be loaded and unloaded on

35 demand so the end-to-end cost stays close to greedy decoding. This is difficult because step-level
 36 calibration must remain reliable across diverse reasoning styles (e.g., chain-of-thought versus program-
 37 of-thought), and because inference-time control must react to local uncertainty while respecting a
 38 global wall-clock budget.

39 1.2 Framework Overview

40 We propose TinyPRM-Cascade, a three-tier inference framework for a 7B generator. First, a Shared-
 41 Adapter PRM ($\sim 400\text{M}$) inserts LoRA adapters into 12 generator layers and reuses the frozen
 42 backbone to yield step-correctness probabilities from a sliding window; a small gating MLP ($\sim 20\text{M}$)
 43 decides whether to activate the adapter path. Second, a Sparse-MoE PRM ($\sim 900\text{M}$ total) distilled
 44 from a 6B teacher routes at token level among 12 experts while activating only the top-2 per call
 45 ($\sim 220\text{M}$ active parameters). Third, an adaptive beam controller observes the generator’s next-token
 46 log-probability gap, the Shared-Adapter PRM score, and a running budget, then either commits
 47 early ($p_{\text{t}} > 0.95$ or $\Delta_{\text{t}} > 0.7$), spawns up to $k \leq 5$ beams with a single MoE-based rerank when
 48 moderately uncertain ($0.65 < p_{\text{t}} < 0.95$), or halts exploration to preserve budget. Distillation
 49 uses a mixture-of-thought corpus combining chain-of-thought and program-of-thought traces for
 50 cross-representation robustness; a calibration head trained with focal Kullback–Leibler down-weights
 51 mislabeled or hallucinated step errors often seen in stepwise evaluators. By sharing parameters with
 52 the generator, the Shared-Adapter PRM adds under 0.5 GB VRAM; the heavier MoE is lazily loaded
 53 only when needed.

54 Our design draws on several threads. Process supervision offers high-quality training signals at
 55 the step level Lightman et al. [2023]. Selection-inference frameworks grow interpretable reasoning
 56 chains Creswell et al. [2022], while self-evaluation guided beams and chain-of-thought decoding
 57 explore search-time guidance, though often at high compute cost or with calibration drift Xie et al.
 58 [2023], Wang and Zhou [2024]. Cascades leveraging mixture-of-thought signals have been effective
 59 for cost-efficient API usage Yue et al. [2023]. We adopt systems primitives such as IO-aware attention
 60 and low-precision inference to reduce latency and memory Dao et al. [2022], Dettmers et al. [2022],
 61 and we are informed by insights from scaling and calibration in large models Bordelon et al. [2024],
 62 Cherti et al. [2022].

63 1.3 Contributions

- 64 • **Parameter-sharing PRM under 0.5 GB VRAM:** Reuses the 7B generator’s backbone via
 65 LoRA adapters and a small gate.
- 66 • **Sparse MoE refinement PRM:** Distilled from a 6B teacher, with token-level routing and
 67 top-2 experts per call, lazily loaded on demand.
- 68 • **Adaptive, budgeted controller:** Fuses PRM scores, log-probability margins, and a wall-
 69 clock budget to decide when to commit, branch, or halt.
- 70 • **Mixture-of-thought distillation and focal KL:** Improves step-level robustness and mitigates
 71 hallucinated errors in evaluators.
- 72 • **Instrumented harness:** Paired testing, bootstrap confidence intervals, and detailed logging
 73 of PRM usage, lazy-load overheads, latency, and memory.

74 1.4 Verification Plan and Current Status

75 We outline three experiments spanning end-to-end cost-accuracy trade-offs, calibration and early-
 76 commit quality, and robustness under stressors, on both datacenter and consumer GPUs. In this paper
 77 we report a complete toy run that validates control flow and measurements. Because greedy decoding
 78 attains 100% accuracy in this toy setting, ceiling effects limit the evaluative power; we analyze these
 79 limitations and motivate full-scale runs on BIG-Bench Hard, GSM8K, SVAMP, and MATH with
 80 PRM800K for step-level metrics Lightman et al. [2023].

81 Looking ahead, we plan hardware-aware gating policies, continual self-distillation during deployment,
 82 and multimodal extensions via shared visual adapters, leveraging efficient attention and quantization
 83 for deployment Dao et al. [2022], Dettmers et al. [2022].

84 2 Related Work

85 2.1 Process Supervision and PRMs

86 Let’s Verify Step by Step demonstrated that process supervision with step-level labels surpasses
87 outcome-only supervision on challenging mathematical reasoning and released PRM800K to support
88 research on intermediate-step correctness Lightman et al. [2023]. TinyPRM-Cascade aligns with this
89 line but targets sub-1B evaluators and tight latency/VRAM budgets via parameter sharing and sparse
90 expert refinement, making process supervision practical at inference time on limited hardware.

91 2.2 Outcome-Level Surrogates for Process Guidance

92 GLoRe introduced stepwise outcome reward models trained on synthetic data to approximate future
93 correctness and to decide when and where to refine Havrilla et al. [2024]. This departs from explicit
94 process supervision by using outcome-based signals; in contrast, our evaluators directly predict step
95 correctness from partial traces and are distilled from a process-supervised teacher, aiming to preserve
96 calibration on intermediate steps Lightman et al. [2023].

97 2.3 Search-Time Guidance and Self-Evaluation

98 Self-Evaluation Guided Beam Search incorporates stepwise self-critique to guide stochastic decoding
99 and improve robustness Xie et al. [2023]. Chain-of-Thought Decoding without prompting reveals
100 latent reasoning paths by adjusting decoding dynamics, avoiding prompt engineering Wang and
101 Zhou [2024]. Our adaptive controller is compatible with these directions but differs by combining a
102 calibrated PRM score with the generator’s margin and an explicit compute budget. It further confines
103 heavier evaluation to uncertainty bands by invoking a sparse MoE only when needed.

104 2.4 Cascades and Mixture-of-Thought

105 Cascades using consistency checks and mixture-of-thought representations can cut costs while
106 matching stronger models’ accuracy Yue et al. [2023]. We adopt the mixture-of-thought insight not as
107 an endpoint selection mechanism but as a distillation corpus to transfer robustness across reasoning
108 styles into compact PRMs.

109 2.5 Beam Search Under Training Constraints

110 Learning under beam-search constraints has been studied to reconcile training and testing objectives
111 and to analyze calibration under beam search Zhuo et al. [2020]. Simulation-guided beam search
112 improves solution quality within fixed compute by combining learned policies with rollouts Choo
113 et al. [2022]. Our method complements these by attaching calibrated evaluators and a budget-aware
114 controller to a frozen generator, rather than retraining the generator to be beam-optimal.

115 2.6 Test-Time Adaptation and Systems Efficiency

116 Test-time training on nearest neighbors enhances language modeling without long-context overheads
117 Hardt and Sun [2023], whereas our focus is on calibrated decision-making during inference without
118 parameter updates. On the systems side, IO-aware attention reduces memory traffic and latency Dao
119 et al. [2022], and 8-bit matrix multiplications cut memory use while retaining accuracy Dettmers et al.
120 [2022]. We leverage these techniques operationally in our harness. Finally, insights from scaling laws
121 and calibration inform our modeling targets and reliability evaluation Bordelon et al. [2024], Cherti
122 et al. [2022].

123 3 Background

124 3.1 Problem Setting

125 Given an input x , a generator G produces a reasoning trace y token-by-token. At step t , a process
126 reward model $S(x, y_{1:t})$ outputs the probability p_{-t} that the partial trace remains on a path to a

correct final answer. The controller observes three signals: (i) the next-token log-probability margin Δ_t between the top two tokens from G ; (ii) the PRM score p_t from S ; and (iii) a running wall-clock budget B derived from a greedy estimate for the same prompt. Based on these, it decides to commit early, branch for refinement, or halt exploration to preserve budget.

3.2 Calibration on Partial Traces

Early commitment requires well-calibrated probabilities over partial traces and robustness across reasoning styles. We distill from a teacher onto a mixture-of-thought set combining chain-of-thought and program-of-thought traces and train a small calibration head with a focal Kullback–Leibler objective that down-weights mislabeled or hallucinated steps, which have been noted in stepwise evaluators trained from synthetic signals. Calibration is evaluated by AUROC, AUPRC, Brier score, and expected calibration error (ECE) on step-labeled data such as PRM800K, alongside step-level F1 Lightman et al. [2023].

3.3 Parameter Sharing and Sparse Experts

To operate under strict memory budgets, the Shared-Adapter PRM reuses G ’s frozen backbone and adds LoRA adapters to a subset of layers. A lightweight gating MLP determines whether the adapter path is needed on a given window of tokens. For harder cases, a Sparse-MoE PRM distilled from a 6B process-supervised teacher uses 12 experts with token-level routing to the top-2, enabling high capacity with low active parameters. Experts are lazily loaded to GPU on demand to avoid persistent memory overhead.

3.4 Assumptions and Notation

We assume access to a process-supervised teacher and step-level labels from PRM800K for training and validation Lightman et al. [2023]. We denote the generator’s confidence margin by Δ_t and the PRM probability by p_t . Controller thresholds use p_{high} around 0.95, p_{low} around 0.65, and Δ_{high} around 0.7 in typical settings. Evaluation uses final-answer accuracy, step-level discrimination and calibration metrics, and resource measures such as tokens, GPU seconds, and VRAM.

3.5 Context Within Prior Art

This formulation connects to selection-inference Creswell et al. [2022] and self-evaluation-guided search Xie et al. [2023], but centers on process-supervised calibration with sub-1B evaluators and compute-aware control. Systems techniques such as IO-aware attention and low-precision inference support deployment under tight latency and memory budgets Dao et al. [2022], Dettmers et al. [2022]. Broader perspectives on scaling and calibration guide modeling choices and evaluation criteria Bordelon et al. [2024], Cherti et al. [2022].

4 Method

4.1 Architecture Overview

TinyPRM-Cascade couples a 7B generator with two evaluators and an adaptive controller.

- **Shared-Adapter PRM (~400M).** We attach LoRA adapters to 12 transformer layers of the frozen generator backbone. For each step, we pool hidden states from a sliding window (either last-token features or a masked average of the last W tokens) and feed them to a small calibration head that outputs p_t , the probability that the partial trace is correct. A 20M-parameter gating MLP consumes the same features to decide whether the adapter path is needed for the current window, allowing most tokens to bypass additional computation.
- **Sparse-MoE Refinement PRM (~900M total, ~220M active).** A router maps pooled features to 12 experts; at inference, the top-2 experts are selected per call. Experts are distilled from a 6B process-supervised teacher on PRM800K plus 60k synthetic mixture-of-thought traces, using temperature-smoothed logits matching and step-level cross-entropy

where gold labels are available. To minimize memory, experts are stored separately and lazily loaded to GPU only when invoked, incurring millisecond-scale overhead per call.

- **Adaptive Beam Controller.** At step t , compute Δ_t from G 's next-token distribution and obtain p_t from the Shared-Adapter PRM. With a global budget B set to 1.5 times a pre-measured greedy decode time for the same prompt, the controller: (1) commits early if $p_t > 0.95$, or $\Delta_t > 0.7$, or if the remaining budget is exhausted; (2) if $0.65 < p_t < 0.95$ and budget allows, spawns up to $k \leq 5$ short rollouts and uses one Sparse-MoE PRM call to rerank their prefixes, then continues with the best branch; (3) otherwise, proceeds greedily without branching. The controller tracks PRM usage and ensures the MoE tier is invoked only under moderate uncertainty.

4.2 Training and Calibration

The Shared-Adapter PRM's calibration head is trained against teacher probabilities with a focal KL objective to reduce sensitivity to mislabeled steps. The gate uses a binary objective derived from an uncertainty heuristic (for example, teacher entropy or margin) and is tuned to minimize unnecessary adapter invocations while preserving accuracy. The MoE PRM is trained with distillation on PRM800K and mixture-of-thought traces, using top-2 routing during training to match inference behavior.

4.3 Computation and Memory

Parameter sharing keeps the PRM under 0.5 GB additional VRAM beyond the 7B generator. The heavier MoE PRM is activated on a minority of steps; experts are loaded from pinned CPU memory, applied for a single scoring pass, and then offloaded. The harness supports IO-aware attention kernels and low-precision inference to reduce latency and memory Dao et al. [2022], Dettmers et al. [2022].

4.4 Algorithmic Summary

For each prompt: (a) estimate greedy decode time; (b) decode with G while, at each step t , computing Δ_t and p_t from the Shared-Adapter PRM over a sliding window; (c) if high confidence or budget exhausted, commit; if moderately uncertain and budget allows, branch with up to k beams, score once with the MoE, and continue with the selected branch; else proceed greedily; (d) stop at EOS or dataset-specific criteria. The harness logs all decisions, PRM and MoE invocations, lazy-load overheads, token counts, peak memory, and GPU seconds for paired statistical analyses.

4.5 Pseudocode: Adaptive Beam Controller

Budget-Aware Adaptive Beam Control **Inputs:** prompt x , generator G , Shared-Adapter PRM S_{SA} , MoE PRM S_{MoE} , thresholds p_{high} , p_{low} , Δ_{high} , beam cap k , budget B Initialize partial trace $y \leftarrow \cdot$, time used $t_{used} \leftarrow 0$ not EOS Obtain next-token distribution $q(\cdot | x, y)$ from G with KV-cache Compute $\Delta_t \leftarrow \log q(\hat{y}_t + 1 | x, y) - \log q(\hat{y}_t + 1 | x, y)$ where $\hat{y}$ and $\tilde{y}$ are top-1 and top-2 tokens Compute $p_t \leftarrow S_{SA}(x, y)$ on sliding window $p_t > p_{high}$ **or** $\Delta_t > \Delta_{high}$ **or** $t_{used} \geq B$ Commit: $y \leftarrow y \cup \{\arg \max q\}$ $p_{low} < p_t < p_{high}$ **and** $t_{used} < B$ Sample up to k rollouts of small horizon h from G conditioned on (x, y) Score prefixes with one call to S_{MoE} ; select best prefix y^* Update $y \leftarrow y^*$ Greedy step: $y \leftarrow y \cup \{\arg \max q\}$ Update t_{used} by measured wall time for this step **Return** final y

4.6 Relation to Prior Techniques

Our controller unifies calibrated stepwise scoring from process supervision with search-time guidance akin to self-evaluation and mixture-of-thought reranking Lightman et al. [2023], Xie et al. [2023], Yue et al. [2023], while explicitly budgeting wall-clock time and minimizing memory via parameter sharing and sparse experts. Efficient attention and quantization provide the systems substrate for practical deployment Dao et al. [2022], Dettmers et al. [2022].

218 5 Experimental Setup

219 5.1 Evaluation Plan

220 We design three experiments: (1) an end-to-end cost-accuracy Pareto comparison against strong
221 baselines; (2) calibration, early-commit quality, and compute allocation; and (3) robustness and
222 on-the-fly PRM loading under resource constraints. Reliability practices include paired evaluation
223 per item, McNemar tests for accuracy, paired bootstrap for latency and token counts, two-run
224 reproducibility with fixed seeds, and 95% BCa confidence intervals.

225 5.2 Datasets and Splits

226 The full plan targets BIG-Bench Hard (23 tasks), GSM8K, SVAMP, MATH (test-mini), and PRM800K
227 dev for step-level metrics Lightman et al. [2023]. In the run reported here, we executed a synthetic
228 toy dataset of character-level arithmetic addition generated by the harness and produced partial traces
229 to probe calibration in a limited fashion.

230 5.3 Models and Baselines

231 The planned system includes a 7B generator fine-tuned with Flan+CoT LoRA, a ~ 400 M Shared-
232 Adapter PRM, and a ~ 900 M Sparse-MoE PRM with 12 experts and top-2 routing. Baselines comprise
233 greedy decoding, self-consistency sampling, a 7B + 6B PRM reranker, and self-evaluation guided
234 beam search Xie et al. [2023], Lightman et al. [2023]. In the reported toy run, we exercised the
235 controller, the Shared-Adapter PRM, and the lazy MoE interfaces; the 6B PRM reranker baseline and
236 VRAM accounting were not executed in this toy setting.

237 5.4 Metrics

238 Primary metrics are final-answer accuracy; median and 95th percentile GPU seconds per example;
239 peak VRAM; and mean tokens. Secondary metrics include PRM call rate, MoE invocation fraction,
240 lazy-load overhead, and step-level calibration metrics (AUROC, AUPRC, Brier, ECE) when labels
241 exist. Robustness probes include adversarial distractions, $3\times$ longer inputs, and noisy retrieval
242 inspired stressors.

243 5.5 Environment and Implementation

244 The harness uses public libraries for transformer inference, parameter-efficient fine-tuning, acceler-
245 ation, and metrics; it enables IO-aware attention and attention key-value caching where available
246 Dao et al. [2022]. Low-precision matrix multiplication reduces memory footprint during inference
247 Dettmers et al. [2022]. All runs fix seeds for determinism. In the reported toy run, wall-clock
248 measurements were recorded; GPU memory statistics and the 6B PRM baseline were not collected.

249 5.6 Ablations

250 We sweep early-commit thresholds ($p_{high} \in \{0.90, 0.95\}$, $p_{low} \in \{0.60, 0.65\}$, $\Delta_{high} \in$
251 $\{0.60, 0.70\}$) and report PRM and MoE usage, tokens, and latency. Planned ablations include
252 removing the MoE tier, disabling adapter sharing, freezing the gate, and varying beam budgets and
253 horizons.

254 5.7 Planned Comparisons

255 Beyond the toy run, we plan to compare TinyPRM-Cascade against self-consistency Xie et al. [2023],
256 chain-of-thought decoding without prompting Wang and Zhou [2024], and cost-efficient cascades
257 with mixture-of-thought Yue et al. [2023], as well as to evaluate calibration on PRM800K Lightman
258 et al. [2023], while leveraging systems baselines for attention and quantization Dao et al. [2022],
259 Dettmers et al. [2022].

6 Results

We report all outputs saved by the harness during a complete toy execution and include all generated figures. The toy task is trivial for the generator, producing ceiling effects that we analyze below.

6.1 Experiment 1: End-to-End Cost-Accuracy Pareto on a Toy Arithmetic Dataset

- **Accuracy:** Greedy 1.00; Self-Consistency 0.05; TinyPRM-Cascade 1.00.
- **Median wall-clock seconds per example:** Greedy 0.000196; Self-Consistency 0.002718; TinyPRM-Cascade 0.000853.
- **PRM usage:** Shared-Adapter PRM mean calls 5.50 per example; MoE calls 0.00 per example.

Interpretation. TinyPRM-Cascade matches greedy accuracy because the task is solved by greedy decoding. It substantially outperforms the self-consistency baseline on both accuracy and latency. Relative to greedy, TinyPRM-Cascade is slower due to per-step PRM scoring overhead at tiny absolute times; this does not reflect the intended operating regime and arises from the toy setup Xie et al. [2023].

6.2 Experiment 2: Calibration and Early-Commit Quality (Toy)

- **Shared-Adapter PRM calibration:** AUROC not defined (degenerate positives/negatives), ECE 0.427, Brier 0.182.
- **MoE PRM calibration:** AUROC not defined (degenerate), ECE 0.050, Brier 0.003.
- **Early-commit sweeps:** For all tested thresholds ($p_{high} \in \{0.90, 0.95\}$, $p_{low} \in \{0.60, 0.65\}$, $\Delta_{high} \in \{0.60, 0.70\}$), final accuracy remained 1.00, with mean PRM calls ≈ 5.47 per example, MoE calls 0.00, and median times around 0.0008-0.0009 seconds.

Interpretation. Degenerate labels yield undefined AUROC and limit the informativeness of ECE and Brier. Because nearly all partial traces are correct on this toy, the MoE is never invoked and early-commit decisions do not materially affect accuracy or compute.

6.3 Experiment 3: Robustness and Budgeted Deployment (Toy)

- **Conditions:** clean, distractors, $3\times$ longer inputs, 5% poisoned retrieval.
- **Accuracies:** all 1.00. Median wall-clock seconds: 0.000915-0.000938.

Interpretation. The toy task is insensitive to these perturbations; the cascade’s lazy-load pathway was not exercised meaningfully.

6.4 Limitations and Fairness

The self-consistency configuration in this toy differs from typical large-scale settings, and the MoE was idle. Peak VRAM and the 7B + 6B PRM reranker comparisons were not collected. As a result, these results cannot support claims about accuracy gains over greedy or $\leq 1.5\times$ latency in realistic scenarios; they only validate control flow and instrumentation Lightman et al. [2023].

6.5 Figures

7 Conclusion

We introduced TinyPRM-Cascade, a compute-aware decoding framework that makes test-time process supervision viable for compact LLMs by combining a parameter-sharing PRM, a lazily loaded sparse MoE refinement PRM distilled from a 6B teacher, and an adaptive beam controller guided by calibrated step scores and generator margins. The design targets sub-1B evaluators, low additional VRAM via parameter sharing, and minimal wall-clock overhead via early commit and on-demand refinement. Our instrumented harness supports rigorous paired evaluation, calibration analysis, and resource accounting.

A complete toy run validates end-to-end control flow, logging, and threshold sweeps, shows dominance over a self-consistency baseline in that setting, and surfaces ceiling effects that limit evaluation. These results are necessary but not sufficient: full-scale experiments on BIG-Bench Hard, GSM8K, SVAMP, and MATH, with PRM800K for step-level calibration, are needed to assess accuracy gains and latency-memory trade-offs Lightman et al. [2023].

7.1 Implications and Future Work

If borne out on realistic benchmarks, sub-1B PRMs can deliver near-state-of-the-art process supervision at a fraction of the latency and memory of 6B-class verifiers, enabling stepwise reasoning on single-GPU desktops and edge servers. Next steps include: (i) completing paired evaluations with confidence intervals and ablations; (ii) implementing hardware-aware gating policies that trade accuracy for latency under tight budgets; (iii) exploring continual self-distillation during deployment; and (iv) extending the cascade to multimodal inputs with shared visual adapters, leveraging efficient attention and quantization to meet deployment constraints Dao et al. [2022], Dettmers et al. [2022].

References

- Blake Bordelon, Alexander Atanasov, and Cengiz Pehlevan. A dynamical model of neural scaling laws. 2024.
- Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 2818-2829, 2022. doi: 10.1109/CVPR52729.2023.00276.
- Jinho Choo, Yeong-Dae Kwon, Jihoon Kim, Jeongwoo Jae, André Hottung, Kevin Tierney, and Youngjune Gwon. Simulation-guided beam search for neural combinatorial optimization. 2022.
- Antonia Creswell, Murray Shanahan, and Irina Higgins. Selection-inference: Exploiting large language models for interpretable logical reasoning. 2022.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. 2022.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. 2022.
- Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. 2023.
- Alex Havrilla, Sharath Rapparthi, Christoforus Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskiy, Eric Hambro, and Roberta Raileanu. Glore: When, where, and how to improve llm reasoning via global and local refinements. 2024.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. 2023.
- Xuezhi Wang and Denny Zhou. Chain-of-thought reasoning without prompting. 2024.
- Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, Xu Zhao, Min-Yen Kan, Junxian He, and Qizhe Xie. Self-evaluation guided beam search for reasoning. 2023.
- Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. Large language model cascades with mixture of thought representations for cost-efficient reasoning. 2023.
- Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. Learning optimal tree models under beam search. 2020.

[width=0.7] images/pareto_accuracy_vs_compute.pdf

Figure 1: Accuracy-compute Pareto on the toy dataset.

[width=0.7] images/accuracy_tinyprm_vs_baselines.pdf

Figure 2: TinyPRM-Cascade versus baselines: accuracy comparison.

[width=0.7] images/inference_latency_tinyprm_vs_baselines.pdf

Figure 3: TinyPRM-Cascade versus baselines: inference latency comparison.

[width=0.7] images/prm_usage_tinyprm.pdf

Figure 4: PRM usage statistics in the cascade.

[width=0.7] images/reliability_curve_shared_adapter.pdf

Figure 5: Reliability curve for the Shared-Adapter PRM.

[width=0.7] images/reliability_curve_moe.pdf

Figure 6: Reliability curve for the MoE PRM.

[width=0.7] images/accuracy_robustness.pdf

Figure 7: Accuracy under robustness stressors.

[width=0.7] images/inference_latency_robustness.pdf

Figure 8: Inference latency under robustness stressors.